

Project Title: Multi-Module Financial Platform Enhancement

Timeline: 6+ months

Role: Product Designer

Team: BA, Development, QA, Stakeholders

Scope: Financial reporting, subscriptions, role architecture, and new time-tracking system (desktop + mobile)

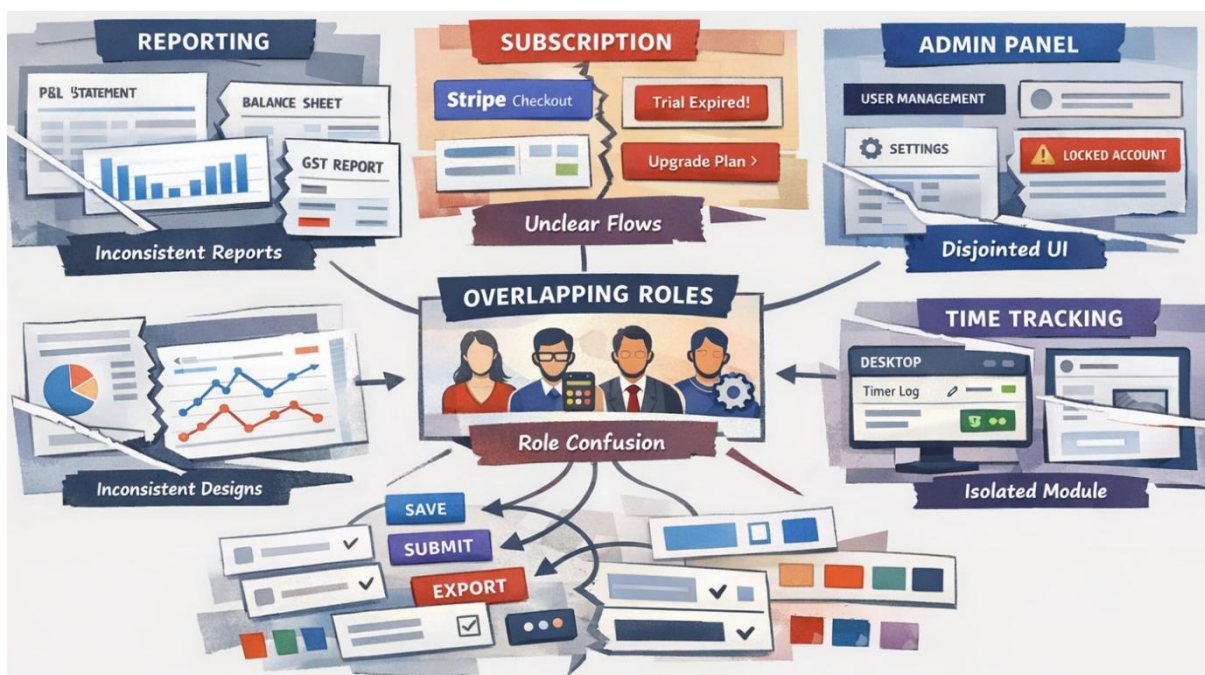
1. Problem

The financial and operations platform was 80% developed but structurally fragmented. As the business expanded, new modules were being layered onto an inconsistent foundation.

Through product audits and stakeholder alignment, I identified five core systemic issues:

1. Fragmented Design System

There was no centralized component governance. UI patterns varied across modules, increasing design debt and slowing development.



2. Inconsistent Reporting Architecture

Financial reports (P&L, Balance Sheet, GST/VAT, Executive Summary) followed different layout structures and filtering logic. Users struggled to:

- Compare reports consistently
- Generate exports efficiently
- Understand financial hierarchy across views

3. Role Ambiguity

Users, accountants, business owners, and admins shared overlapping but unclear interfaces.

Role Confusion Matrix			
✖ = No access ⚠ = Partial / Undefined / Inconsistently implemented ✅ = Full access			
Role	Can View	Can Edit	Admin Actions
General User	Financial Reports Time Logs Subscription Status	Time Logs Profile Info	✖ None
Accountant	Financial Reports GL Codes Transaction Logs Time Logs	Manual Journals GL Mapping Adjust Reports	⚠ Export Financial Data
Business Owner	All Financial Reports Subscription Data User Activity	Approve Journals Edit Tax Settings Subscription Plan	⚠ Manage Users
Admin	Full System Access	All Modules User Permissions Tax Configuration	✅ User Creation/Deletion System Overrides Subscription Overrides

This led to:

- Permission confusion

Overlap Heat View				
Capability	User	Accountant	Owner	Admin
View Reports	✅	✅	✅	✅
Edit Financial Data	✖	⚠	⚠	✅
Export Reports	✖	⚠	✅	✅
Manage Users	✖	✖	⚠	✅
Subscription Override	✖	✖	⚠	✅

- Increased support requests
- Cognitive overload in complex workflows

4. Incomplete Subscription Lifecycle UX

Stripe checkout existed, but:

- Retention flows were undefined
- Lockout states lacked clarity
- Account closure and trial logic were inconsistent

5. Expansion Risk

The company planned to launch a time-tracking module.

Without structural alignment, this expansion would significantly increase long-term design debt.

Core Product Risk:

The system was scaling in features, but not in architecture.

2. Constraints

Constraint	Risk Introduced	Design Response
Legacy Backend Architecture	Could not restructure financial data models or reporting logic. Risk of surface-level redesign without structural fix.	Designed structured GL dropdown system aligned to existing schema instead of backend refactor. Standardized front-end reporting templates to create consistency without breaking backend logic.
Partially Implemented Stripe API	Billing flows already coded. Rewriting logic would delay release and increase engineering cost.	Maintained core Stripe structure. Designed complete lifecycle UX around it (trial logic, lockout messaging, retention flows, admin overrides).
Ongoing Development Sprints	No opportunity for full platform rebuild. High risk of fragmented releases.	Phased rollout strategy. Introduced modular design system governance that could be layered incrementally without halting production.
Financial Compliance Requirements (GL mapping, audit traceability)	Errors could impact financial accuracy and regulatory compliance.	Replaced manual GL inputs with structured dropdown selection tied to backend codes. Standardized report hierarchy to improve audit clarity.
Multiple Stakeholder Groups with Competing Priorities	Conflicting expectations across Finance, Operations, and Admin teams.	Conducted stakeholder alignment workshops. Introduced role-based architecture to clearly separate financial, operational, and administrative workflows.
Desktop-First Financial Users vs Mobile-First Time Tracking Users	Risk of inconsistent UX patterns and fractured module expansion.	Built a unified component system in Figma. Maintained shared navigation logic and interaction patterns across desktop financial modules and mobile time tracking.
Expansion: New Time Tracking Module	Adding new module onto unstable foundation would multiply design debt.	Designed time tracking using the same reporting logic, export patterns, and role architecture to ensure ecosystem coherence.

This redesign was executed under real-world constraints:

- Legacy backend architecture (limited ability to restructure data models)
- Stripe API logic already partially implemented
- Ongoing development sprints (no full rebuild opportunity)
- Financial compliance requirements (GL mapping, audit traceability)
- Multiple stakeholder groups with competing priorities
- Need to support desktop-first financial users and mobile-first time tracking users

This was not a greenfield redesign as it required structural alignment without breaking production systems.

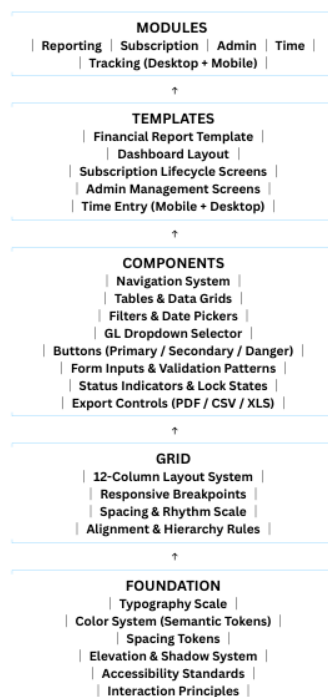
3. Hypothesis

If I rebuilt the platform around a scalable design system and clarified role-based workflows, then:

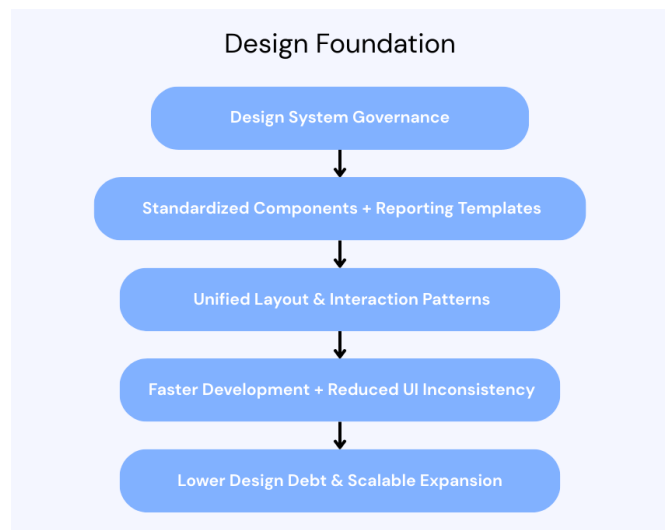
- Financial report generation would become faster and more intuitive
- Manual GL input errors would decrease
- Subscription lifecycle confusion would reduce churn and support load
- Admin workflows would become more efficient
- New modules (time tracking) could scale without increasing complexity

The strategic goal was not cosmetic redesign - it was structural product alignment.

4. Design Decisions



A. Rebuilding the Design Foundation

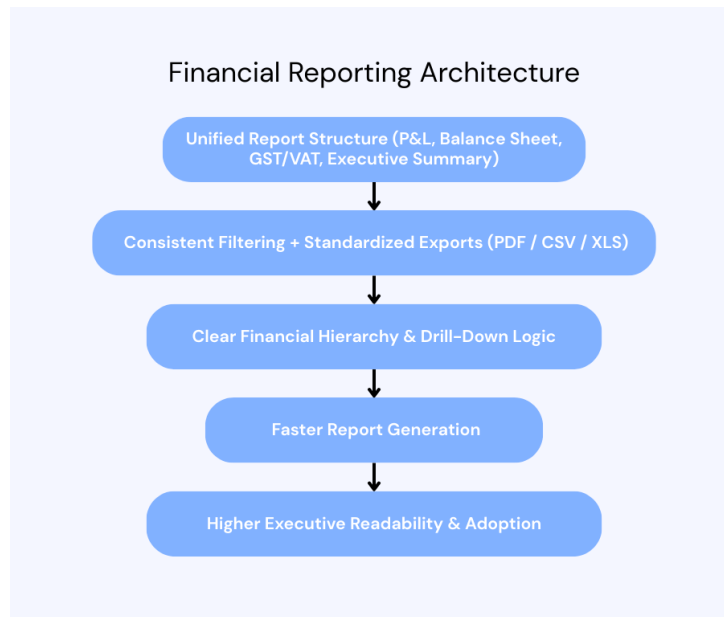


I conducted a full design audit and rebuilt the system in Figma:

- Standardized grid and spacing system
- Consolidated reusable components
- Defined reporting templates
- Created governance for scalable module expansion

This ensured new features aligned with a unified framework rather than creating further inconsistency.

B. Financial Reporting Architecture Redesign



Structured Report System

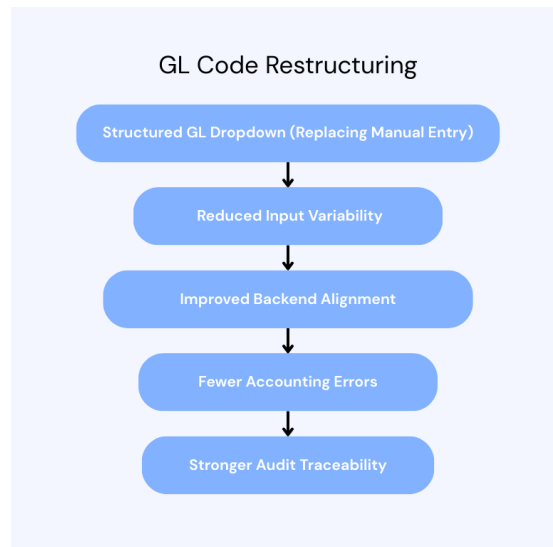
Designed unified templates for:

- Profit & Loss
- Balance Sheet
- GST/VAT
- Executive Summary
- Aged Payables
- Transaction Reports

Key Improvements:

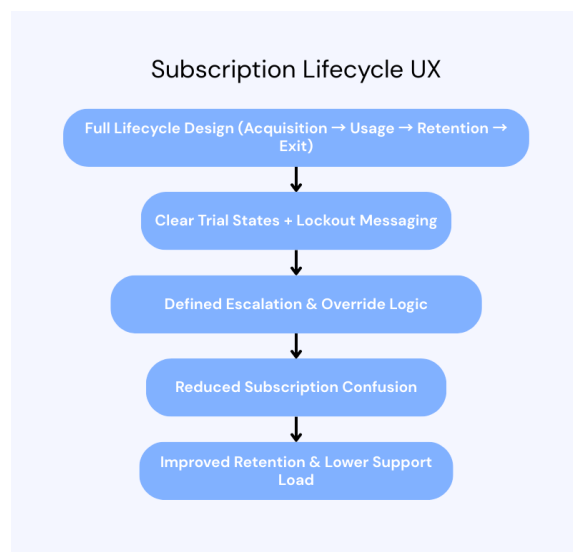
- Unified date-range filtering logic
- Standardized export functionality (PDF / CSV / XLS)
- Clear financial hierarchy for executive readability
- Consistent header, summary, and drill-down patterns
- GL Code Integration

Replaced manual GL entry with dropdown-based structured selection to:



- Reduce accounting errors
- Align with backend financial logic
- Improve audit traceability

C. Subscription & Lifecycle UX



Redesigned subscription flow beyond checkout:

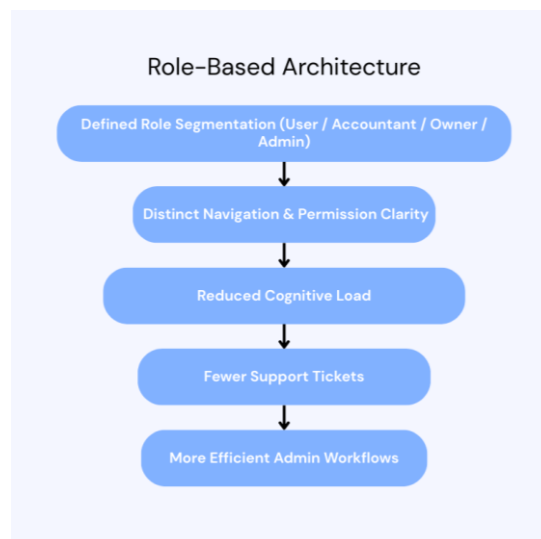
- Structured pricing hierarchy
- Free trial clarity
- Payment settings dashboard
- Lockout state messaging
- Account closure & retention flows

Admin-facing improvements:

- Coupon management
- Subscription overrides
- Email template management

The system now supported the full lifecycle: acquisition, usage, retention, and exit.

D. Role-Based Architecture



Defined distinct UX patterns for:

- General Users
- Accountants
- Business Owners
- Admins

Accountant Portal:

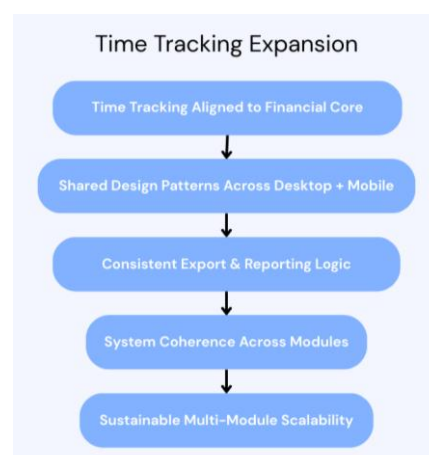
- Workpapers
- Manual Journals
- Fixed Assets
- Year-End Reporting

Admin Portal:

- Multi-user management
- Tax configuration
- Migration tools

Each role received tailored navigation, reducing cognitive load and permission ambiguity.

E. Expansion: Time Tracking System (Desktop + Mobile)



Designed a new workforce time-tracking module aligned with the financial core.

Role Structure

- Owners → Workspace creation
- Coaches → Team management
- Athletes → Log time
- Admins → Review & export

Mobile Focus

- Quick start/stop logging
- Break tracking
- Minimal friction entry

Desktop Focus

- Approval workflows
- Cumulative reporting
- Export capabilities

Maintained consistent design patterns with financial modules to ensure ecosystem coherence.

5. Tradeoffs

This project required strategic compromises:

- Could not redesign backend reporting logic fully due to legacy constraints
- Maintained Stripe's core API structure rather than building custom billing logic
- Prioritized structural consistency over experimental UI innovation
- Phased release approach instead of full platform relaunch
- Balancing ambition with technical feasibility was critical.

6. Outcome

The redesign resulted in measurable product improvements:

Reporting

- Reduced financial report generation time
- Increased report export usage rate
- Improved executive readability across reports

Data Accuracy

- Reduced manual GL entry errors through structured dropdown integration

Subscription

- Improved subscription retention through lifecycle clarity
- Reduced subscription-related support tickets

Admin Efficiency

- Reduced admin task complexity through clearer role segmentation
- Decreased workflow step count in user management flows

System-Level Impact

- Unified multi-module design system
- Reduced long-term design debt
- Scalable architecture for future expansion
- Successful launch of mobile time tracking

7. Key Learnings

- Financial products require structural logic before visual refinement.
- Role-based clarity significantly reduces cognitive load in enterprise systems.
- Subscription UX must account for the full lifecycle, not just checkout.
- Scaling products demands governance, not just feature expansion.
- Cross-functional alignment is essential in long-cycle system redesigns.